1. Цель MVP

Первый MVP PaperWeave — **работающий каркас** системы:

```
Браузер → NiceGUI → FastAPI → PostgreSQL
```

Пользователь открывает интерфейс, видит состояние API, создаёт исследовательский проект и просматривает список проектов. Данные сохраняются между перезапусками.

Изменения относительно предыдущих требований:

- нестандартные порты вместо 8000 и 8080;
- настройки приложения через конфигурационный файл;
- минимальный `Makefile` для управления проектом;
- конфигурация, миграции и запуск входят в первую реализацию.

2. Стек и границы

Компонент	Решение
Backend	Python + FastAPI + Pydantic
Frontend	NiceGUI, отдельный процесс
HTTP-клиент GUI	httpx, асинхронный
БД	PostgreSQL
Работа с БД	SQLAlchemy + psycopg
Миграции	Alembic
Конфигурация	TOML-файл
Запуск	Docker Compose
Команды разработки	Makefile
Тестирование	pytest

В MVP не входят:

- загрузка и разбор PDF;
- LLM-интеграция;
- embeddings и pgvector;

- фоновые задания;
- блочный редактор;
- авторизация и публичное размещение.

3. Порты и доступ

Фиксируем следующие значения по умолчанию:

Сервис	Порт	Адрес на хосте
Backend	`18741`	`http://127.0.0.1:18741`
Frontend	`18742`	`http://127.0.0.1:18742`
PostgreSQL	`5432`, только внутри Compose	Не публикуется

Все порты приложения задаются конфигурацией, а не зашиваются в код.

Backend и GUI слушают `0.0.0.0` внутри контейнеров, но публикуются на `127.0.0.1` хоста. Доступ из внешней сети в этот MVP не закладываем.

Выбранные порты — значения по умолчанию, не гарантия их свободности на конкретной машине.

4. Конфигурация через файл

Файлы

```
config.example.toml    # Шаблон, хранится в Git
config.toml            # Локальные настройки, исключён из Git
```

`.env` для настроек приложения не используем.

Предлагаемое содержимое `config.example.toml`:

```
[app]
name = "paperweave"
log_level = "INFO"

[api]
host = "0.0.0.0"
port = 18741

[ui]
host = "0.0.0.0"
port = 18742
api_base_url = "http://api:18741"

[http]
health_timeout_seconds = 2
request_timeout_seconds = 10
```

```
health_poll_interval_seconds = 10

[database]
host = "db"
port = 5432
name = "paperweave"
user = "paperweave_app"
password = "change_me"

[deployment]
bind_host = "127.0.0.1"
```

Требования к загрузке настроек

- Конфигурация загружается при старте каждого процесса.
- Отсутствующий или некорректный файл приводит к завершению с понятной ошибкой.
- Проверяются типы, обязательные поля, диапазоны портов и положительные таймауты.
- Пароль БД и полная строка подключения не выводятся в логи.
- Изменения конфигурации применяются после перезапуска.
- Путь к файлу можно передать аргументом `--config`; по умолчанию — `config.toml`.

Связь с Docker Compose

Единый источник настроек — `config.toml`.

Для минимальной реализации предусматриваем небольшой launcher на Python, который:

1. читает и валидирует TOML;
2. передаёт необходимые значения Docker Compose через окружение дочернего процесса;
3. запускает Compose;
4. монтирует тот же конфигурационный файл в контейнеры в режиме read-only.

Таким образом, не требуется вручную синхронизировать порты и реквизиты БД между TOML, `.env` и `compose.yaml`. Служебные переменные процесса Compose не являются отдельным пользовательским конфигом.

5. Минимальный Makefile

Основной способ работы с проектом — команды `make`.

Команда	Действие
`make init`	Создать `config.toml` из шаблона, если файла ещё нет

Команда	Действие
`make config-check`	Проверить конфигурацию без запуска сервисов
`make up`	Проверить конфиг, собрать образы, выполнить миграции и запустить сервисы
`make down`	Остановить окружение без удаления данных
`make restart`	Пересоздать процессы приложения с актуальной конфигурацией
`make logs`	Показать логи сервисов
`make ps`	Показать состояние контейнеров
`make migrate`	Выполнить миграции вручную
`make test`	Запустить тесты в изолированном тестовом окружении
`make help`	Показать список команд

Первый запуск:

```
make init
# При необходимости изменить config.toml
make up
```

Требования:

- `make init` не перезаписывает существующий конфиг.
- `make up` допускает повторный запуск.
- Ошибки конфигурации и миграций возвращают ненулевой exit code.
- `make down` не удаляет volume PostgreSQL.
- Команду удаления данных в минимальный Makefile не добавляем.
- Тесты не используют рабочую БД.

6. Старт сервисов

Состав окружения:

Сервис	Назначение
`db`	PostgreSQL с постоянным volume
`migrate`	Однократное выполнение `alembic upgrade head`
`api`	FastAPI
`ui`	NiceGUI

Порядок запуска:

```
Проверка config.toml
        ↓
```

```
PostgreSQL готова принимать подключения
        ↓
Миграции успешно завершены
        ↓
Backend запущен
        ↓
Frontend запущен
```

Если миграция завершилась с ошибкой, приложение не должно запускаться как будто окружение готово.

После запуска доступны:

```
GUI:       http://127.0.0.1:18742
API docs:  http://127.0.0.1:18741/docs
```

7. Минимальные API-точки

Префикс: `/api/v1`.

Метод	Endpoint	Результат
GET	`/health/live`	Проверка работы процесса
GET	`/health/ready`	Проверка доступности БД
GET	`/projects`	Список проектов
POST	`/projects`	Создание проекта
GET	`/projects/{project_id}`	Получение проекта

Health

`GET /health/live` — 200, без обращения к БД:

```
{
  "status": "ok",
  "service": "paperweave-api"
}
```

`GET /health/ready` — 200, если БД доступна:

```
{
  "status": "ready",
  "checks": {
    "database": "ok"
  }
}
```

Если БД недоступна — 503:

```
{
  "status": "not_ready",
  "checks": {
    "database": "unavailable"
  }
}
```

Проекты

Запрос `POST /projects`:

```
{
  "name": "Обзор RAG-систем",
  "description": "Сопоставление подходов к работе с источниками"
}
```

Правила:

- `name`: обязательное, от 1 до 200 символов после удаления крайних пробелов;
- `description`: необязательное, до 5000 символов;
- одинаковые названия разрешены;
- ID и время создания задаёт backend.

Успешный ответ — `201`, объект проекта с полями:

```
id
name
description
created_at
```

`GET /projects` возвращает:

```
{
  "items": []
}
```

Для непустого списка — объекты проектов, отсортированные от новых к старым.

`GET /projects/{project_id}`:

- `200` — найден;
- `404` — отсутствует;
- `422` — некорректный UUID.

Редактирование, удаление и пагинация пока не входят в контракт.

8. Минимальная БД

Одна прикладная таблица:

```
projects
├── id            UUID, primary key
├── name          VARCHAR(200), not null
├── description   TEXT, nullable
└── created_at    TIMESTAMPTZ, not null
```

Дополнительно создаётся служебная таблица Alembic.

Даты API возвращает в UTC. GUI не обращается к PostgreSQL напрямую — только через API.

9. Frontend и опрос API

Главная страница

- название приложения;
- статус backend;
- форма создания проекта;
- список проектов;
- кнопка «Обновить»;
- переход на страницу проекта.

Страница проекта

- название;
- описание;
- ID;
- дата создания;
- возврат к списку.

Запросы

Событие	Действие
Открытие главной страницы	`GET /health/ready` и `GET /projects`
Каждые 10 секунд	`GET /health/ready`
Создание проекта	`POST /projects`
Успешное создание	Повторный `GET /projects`
Нажатие «Обновить»	`GET /projects`

Событие	Действие
Открытие проекта	`GET /projects/{id}`

Интервал и таймауты берутся из `config.toml`.

Требования к поведению:

- запросы асинхронные и не блокируют интерфейс;
- health-проверки не перекрываются;
- после закрытия страницы таймер прекращает работу;
- при отправке формы кнопка создания блокируется;
- при ошибке введённые данные сохраняются;
- автоматического повтора `POST` нет;
- недоступность API не приводит к падению GUI.

Статусы:

```
API: проверка…
API: доступен
API: не готов
API: нет соединения
```

10. Минимальная структура репозитория

```
paperweave/
├── Makefile
├── compose.yaml
├── Dockerfile
├── pyproject.toml
├── uv.lock
├── config.example.toml
├── .gitignore
├── README.md
├── alembic.ini
│
├── scripts/
│   └── compose.py
│
├── src/paperweave/
│   ├── config.py
│   ├── api/
│   ├── ui/
│   ├── modules/projects/
│   └── infrastructure/db/
│
├── migrations/
└── tests/
```

```
    ├── unit/
    └── integration/
```

`config.toml`, файлы БД и секреты в Git не попадают.

11. Критерии приёмки

MVP готов, если:

1. `make init` создаёт конфиг, не перезаписывая существующий.
2. `make config-check` обнаруживает ошибки настроек.
3. `make up` собирает и запускает окружение с миграциями.
4. API и GUI работают на `18741` и `18742`, порт `8000` не используется.
5. Смена портов в конфиге применяется без правки исходников и `compose.yaml`.
6. GUI показывает доступность backend и позволяет создать проект.
7. Проект сохраняется после `make down` и повторного `make up`.
8. При остановке API GUI показывает отсутствие соединения.
9. При недоступной БД readiness возвращает `503`.
10. `make test` проверяет конфигурацию и основные операции с проектами, не затрагивая рабочие данные.